

An Energy-Efficient High-Utilization Hardware Architecture for Attention Mechanism in Transformer using Balanced Systolic Array and Multi-Row Interleaved Operation Ordering

Haiyang Zhou^{1,2}, Hongyang Hu^{1*}, Jinshan Yue¹, Hanghang Gao^{1,2}, Yuanlu Xie¹, Xiaoxin Xu¹, Chunmeng Dou¹, Ming Liu³

¹Key Laboratory of Fabrication Technologies for Integrated Circuits, Institute of Microelectronics of Chinese Academy of Sciences, Beijing, China, ²University of Chinese Academy of Sciences, Beijing, China, ³Frontier Institute of Chip and System, Fudan University, Shanghai, China

*Corresponding author: huhongyang@ime.ac.cn

Abstract—Transformer-based neural networks have achieved remarkable performance. Designing energy-efficient and high-speed accelerators for the attention mechanism, which dominates the energy and latency in Transformers, has become increasingly significant. Existing attention accelerators commonly use algorithm-hardware co-design to achieve higher energy efficiency and speed. However, deeply customized algorithms make these accelerators dependent on a particular application. Therefore, optimizing hardware architecture is crucial for achieving general-purpose acceleration. We observe two limitations in the hardware architecture of existing attention accelerators. First, the widely used input stationary, weight stationary, and output stationary systolic arrays (SAs) can't balance data reuse, register saving, and utilization, which hinders to build more energy-efficient and faster SA-based accelerators. Second, layer-by-layer operation ordering introduces high SRAM access overhead of intermediate results. To address the first limitation, we propose the "Balanced Systolic Array", which improves energy efficiency by 40% compared to conventional systolic arrays and achieves a utilization rate of 99.5%. To address the second limitation, we propose "Multi-Row Interleaved" operation ordering, which reduces the SRAM energy by 31.7%. By integrating two techniques, the proposed attention accelerator achieves a 39% improvement in energy efficiency and a 38% enhancement in throughput×energy efficiency compared to previous works.

I. INTRODUCTION

Transformer-based neural networks have demonstrated superior model performance over RNNs and CNNs in fields such as natural language processing[1][2][3] and computer vision[4]. In Transformers, the attention mechanism dominates the energy and latency[5][6][7]. Therefore, designing energy-efficient and high-speed accelerators for the attention mechanism has become increasingly significant. The computation process of the attention mechanism is shown in Fig.1, where n represents the token length, and d represents the word embedding dimension. The attention score matrix S is obtained by multiplying Q with K^T . After S is dequantized, scaled, processed with Softmax, and quantified, the attention probability matrix P is generated. Finally, P is multiplied by the V to generate the output matrix Z . The data formats for matrix

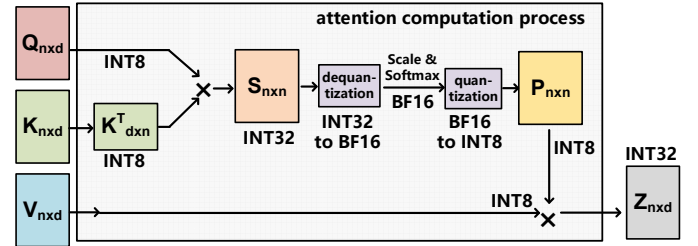
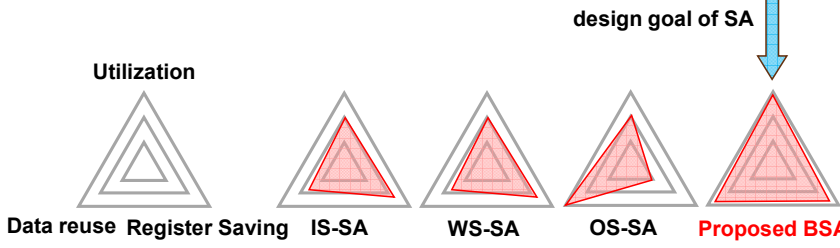


Fig. 1. Computation process of attention mechanism.

multiplication(MM), accumulation, Softmax are INT8, INT32, and BF16, respectively[17]. Existing attention accelerators commonly use algorithm-hardware co-design to achieve higher energy efficiency and speed. [7][9][10][11][12] exploit matrix P sparsity to introduce sparse computing, while [5][6][8] exploit the weak relevances tokens to introduce approximate computing. However, deeply customized algorithms tie them to a particular application[13]. Therefore, focusing on hardware architecture optimizations alone is vital for general-purpose acceleration.

We observed two hardware architecture limitations in existing attention accelerators, which introduce new challenges. As shown in Fig.2, **the first limitation** is the widely used input systolic (IS), weight stationary (WS), and output stationary (OS) SAs fail to balance data reuse, register saving, and utilization. Data reuse and register saving affect energy efficiency, as they affect the energy of memory[15], and register write energy[16]. Utilization impacts speed, as it affects throughput[13]. Therefore, the limitation restricts to build more energy-efficient and higher-speed SA-based accelerators. Specifically, for data reuse, OS-SA maximizes data reuse by using the outer product for MM[10][18], while IS-SA (or WS-SA) has lower data reuse, as it uses the inner product for MM. For register saving, OS-SA requires registers for accumulation and systolic, whereas IS-SA and WS-SA only need registers for systolic. For utilization, SAs require preloading data (IS-SA or WS-SA), offloading results (OS-SA), and filling pipeline(IS-SA, WS-SA, OS-SA), all of which introduce idle cycles that reduce utilization.

Limitation 1 : Widely used IS/WS/OS Systolic array can't balance data reuse, register saving, utilization.



Limitation 2 : E_{P_sram} dominates the E_{sram} in the layer-by-layer operation ordering.

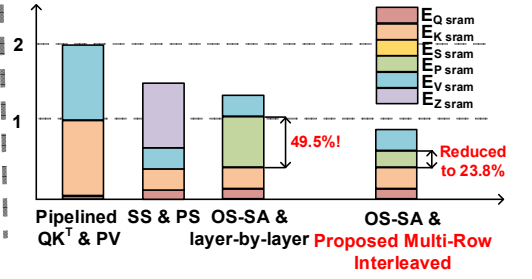


Fig. 2. Two Limitations of hardware architecture in existing attention accelerators.

Therefore, designing an SA that balances data reuse, register saving, and utilization introduces the first challenge.

The second limitation is the layer-by-layer operation ordering introduces significant SRAM access overhead of intermediate results. The right side of Fig.2 shows the total SRAM energy profile of the previous attention accelerator with different architectures (E_{sram} , unit: J, $E_{sram} = E_{Q_sram} + E_{K_sram} + E_{sram} + E_{P_sram} + E_{V_sram} + E_{Z_sram}$). The first architecture pipelines $Q \times K^T$ and $P \times V$ (pipelined QK^T & PV)[5][6][7]. Although this architecture eliminates SRAM access of intermediate results ($E_{S,P_sram} = 0$), it weakens the spatial reuse of $Q \times K^T$ and $P \times V$ since the multipliers are split into two arrays, resulting in the highest E_{sram} . The second architecture uses reconfigurable SA which configures OS-SA when $Q \times K^T$ and IS-SA when $P \times V$ (S stationary & P stationary, SS & PS)[8][10]. Although it not only avoids multiplier splitting but also eliminates SRAM access of intermediate results, the shape of the array for optimal data reuse for IS-SA and OS-SA is different, and IS-SA introduces inner product that weakens data reuse. The third architecture uses OS-SA and layer-by-layer operation ordering to compute $Q \times K^T$ and $P \times V$ (OS-SA & layer-by-layer)[12]. Although it not only avoids multiplier splitting but also remains the optimal shape of the array for data reuse, the layer-by-layer operation ordering introduces large energy of SRAM access of intermediate results (E_{P_sram} accounts for 49.5%). Therefore, designing an operation ordering to reduce E_{P_sram} introduces the second challenge. To address the above challenges, we made three contributions:

- We propose the "Balanced Systolic Array" to address the first challenge, which balances data reuse, register saving, and utilization, further improving energy efficiency and speed of SA.
- We propose a mathematical formula to determine the optimal data reuse for OS-based arrays with the inner-outer-mixed product.
- We propose the "Multi-Row Interleaved" operation ordering to address the second challenge, which significantly reduces E_{P_sram} , further improving energy efficiency for attention accelerators.

The rest of this paper is organized as follows: Section II presents the background on three approaches for matrix multiplication and systolic array. Section III details the proposed attention accelerator architecture, including the "Balanced

Systolic Array" and "Multi-row Interleaved" operation ordering. Section IV compares different systolic arrays and architecture of attention accelerators in terms of energy, energy efficiency, and utilization. Section V concludes the paper.

II. BACKGROUND

A. Approaches for Matrix Multiplication (MM)

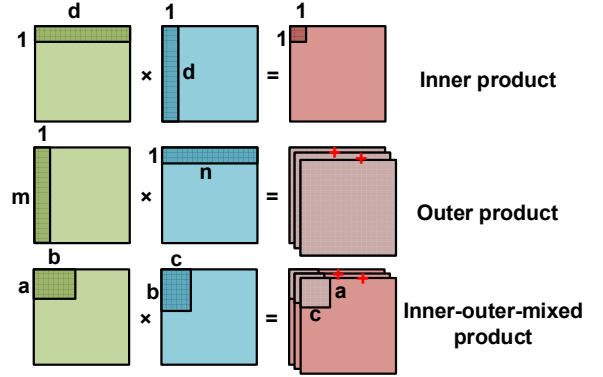


Fig. 3. Three approaches for matrix multiplication computation.

Fig. 3 shows three approaches for MM of an $m \times d$ matrix and a $d \times n$ matrix. In the output matrix, intensified red represents the result, while gray represents partial sums (psum).

The inner product approach is characterized by obtaining an element of the output matrix by taking the dot product of a row of the $m \times d$ matrix with one column of the $d \times n$ matrix. In hardware implementation, the inner product is characterized by two or more psums required for one output element to be generated in parallel. Outer product approach which is optimal in input reuse and output reuse, resulting in the highest data reuse[18], where a column of the $m \times d$ matrix is multiplied by a row of the $d \times n$ matrix to yield a psum matrix. Summing all psum matrices produces the final output matrix. In hardware implementation, the outer product is characterized by the psums required for one output element that are not generated in parallel, and the elements of both $m \times d$ matrix and $d \times n$ matrix are spatially reused by multiple multipliers. Inner-outer-mixed product approach generates a psum matrix block by multiplying a block from the $m \times d$ matrix and a block from the $d \times n$ matrix. Summing all corresponding psum matrix blocks yields the final output matrix. In hardware implementation, it has characteristics both of inner product and outer product.

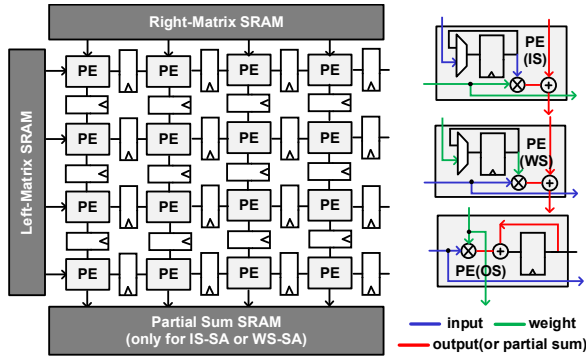


Fig. 4. Typical architecture of systolic array.

B. Conventional Systolic Array

Fig.4 shows a typical systolic array(SA) and specific PE structures of IS-SA, WS-SA, and OS-SA. SAs consist of many homogeneous processing elements (PEs), and data is transmitted between PEs through registers, thereby achieving spatial reuse. In IS-SA (or WS-SA), inputs (or weights) need to be preloaded into the registers of the PEs and then latched. After that, weights (or inputs) flow horizontally to the SA, while psums are propagated and summed vertically. The pipeline is filled when weights(or inputs) reach all PEs. In OS-SA, there is no preload process. Inputs flow horizontally to the OS-SA, weights flow vertically to the OS-SA, and each PE accumulates psum locally. Once inputs and weights reach all PEs, the pipeline is filled. The matrix multiplication result is offloaded through accumulators in OS-SA.

In our proposed systolic array, we combine the advantages of good data reuse of the outer product and register saving of the inner product by using the inner-outer-mixed product. We also combine the good register saving of broadcast and the good scalability of systolic. Furthermore, we improve the utilization of the SA by overlapping computation and offload.

III. PROPOSED HARDWARE ARCHITECTURE

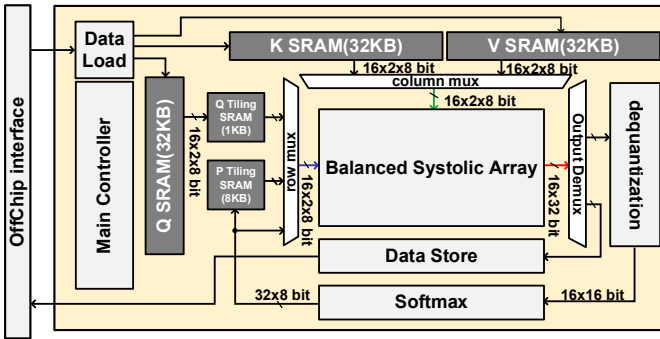


Fig. 5. Top-level of proposed attention accelerator architecture.

A. Overview

Fig. 5 shows the top-level architecture of the proposed attention accelerator. The memory mainly includes Q, K, V, Q Tiling, and P Tiling SRAMs. The Q, K, and V SRAMs store the Q, K, and V data, which are loaded by the Data Load module.

The computational components consist of a balanced systolic array (BSA) and the Softmax module. The row mux and column mux (col mux) dynamically select the source data to feed into the BSA. The BSA contains 512 multipliers and 512 adders, with INT8 for the multipliers and INT32 for the accumulators. Two input matrices, represented by blue and green lines, are fed into the BSA, and the matrix multiplication results are output along the red line. The $Q \times K^T$ result S is fed into a dequantization module, while the $P \times V$ result Z is output via the Data Store module to the chip's I/O. The Softmax module has 16 exponent function units in BF16 format, which receives the data from the dequantization module and generates the P data. The Q Tiling SRAM and P Tiling SRAM enable data tiling for Q and P data, sourced from Q SRAM and the P data from the Softmax module. The $Q \times K^T$ and $P \times V$ are time-interleaved computed on the BSA.

B. Proposed Balanced Systolic Array

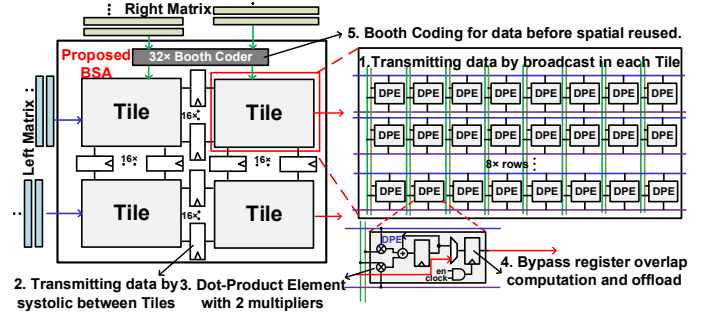


Fig. 6. The structure of the proposed Balanced Systolic Array.

Fig.6 shows the structure of the balanced systolic array which has three features.

Feature1: Inner-Outer-Mixed Product Broadcast Systolic Array (IOM-BSA, labeled 1, 2, 3 in Fig.6). IOM-BSA improves energy efficiency by balancing data reuse and register saving which is a variant of OS-SA. In IOM-BSA, each element of the left (or right) matrix is spatially reused by multipliers in the same row(or in the same column), which achieves outer product to obtain good data reuse. The array is divided into four tiles. In each tile, using broadcast to save registers for systolic and reduce systolic cycles which reduces pipeline filling cycles. Between tile and tile, using registers to transmit data, maintaining the scalability of the systolic array. Each tile contains an 8×8 array of Dot-Product Element (DPE) units. The dot-product operation achieves inner product to save accumulators compared to OS-SA. In summary, the shape of the array is set to 16×16 units, each with two multipliers. We have two considerations for setting the number of multipliers in DPE to 2.

Proposed mathematical formula: In order to determine the optimal data reuse for OS-based arrays with the inner-outer-mixed product, we discuss matrix multiplication $C_{m \times n} = A_{m \times d} B_{d \times n}$, and consider that the data format of the multiplier and accumulator are INT8, INT32, respectively. The SRAM for storing A, B, and Psum (or C) are denoted as A SRAM, B SRAM, and Psum SRAM. Therefore, if an array has

no spatial reuse at all, then it needs to access $m \times n \times d \times 8$ bits from A SRAM(read), $m \times n \times d \times 8$ bits from B SRAM(read), and $2 \times m \times n \times d \times 32$ bits from Psum SRAM(write and read). In addition, we denoted the number of rows of the array as a (B is spatially reused a times), the number of multipliers in a unit as b (P_{sum} is spatially reused b times), and the number of columns as c (A is spatially reused c times). Then when performing MM, the access of A SRAM is $8mnd/c$ bits. The access of B SRAM is $8mnd/a$ bits. The psum memory access is 0 since output-stationary. Therefore, the total SRAM access is:

$$\frac{8mnd}{a} + \frac{8mnd}{c} \quad (1)$$

Since $a \times b \times c = \text{number of multipliers}$, substituting $abc=512$ into the formula(1) leads to a basic inequality, as shown in formula(2).

$$\frac{8mnd}{a} + \frac{mnd}{64} ab \geq \frac{mnd \sqrt{b}}{\sqrt{2}} \quad (\text{when } a = \frac{16\sqrt{2}}{\sqrt{b}}, c = \frac{32}{\sqrt{2}\sqrt{b}}) \quad (2)$$

Formula(2) shows that as b increases, data reuse decreases. Therefore, to maintain good data reuse, we prefer to select $b=2$. At the same time, we can determine the optimal shape of the array for data reuse is 16 rows and 16 columns.

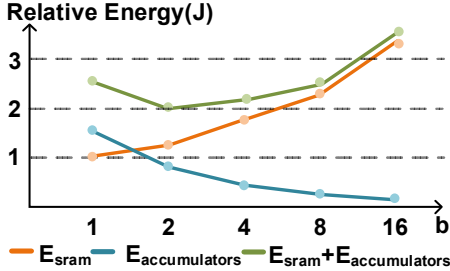


Fig. 7. Energy of SRAM and accumulators versus b .

As shown in Fig.7, to further balance data reuse and register saving, we evaluated the energy curve of SRAM (E_{sram}) and the energy curve of the accumulator ($E_{accumulator}$) as b increases while computing $Q \times K^T$. $E_{sram} + E_{accumulator}$ reaches its minimum value when $b = 2$. Therefore, we set two multipliers in each DPE based on formula(2) and Fig.7.

Feature 2: Bypass Register (labeled 4 in Fig.6) to balance utilization. Each DPE contains a bypass register with three behaviors: 1) Copying the value of the accumulator in each DPE, 2) Transmitting the bypass register value from the left DPE to the right DPE, and 3) Gating. The offloading task is accomplished through the bypass register by copying the accumulator result to the bypass register, allowing computation and offloading to overlap. Utilization is improved since the latency caused by offloading is hidden. The bypass register only works during the offload phase and is turned off via clock gating at other times. The additional register write energy due to the bypass register is negligible, as offload cycles are much less than computation cycles.

Feature 3: Booth Sharing (labeled 5 in Fig.6). Booth Sharing further improves energy efficiency. In conventional matrix multiplication arrays, each INT8 multiplier typically contains a Booth encoder circuit to implement a high-speed

multiplier. We observed that the outer-product-based array shares the same operands across rows and columns, which introduces duplicate Booth coding. Therefore, we encode the operands in each column once and then spatially reuse the encoded operands, reducing the number of Booth encoder circuits. Booth Sharing decreases multiplier energy, improving energy efficiency. Specifically, we reduced the number of Booth encoder circuits from 512 to 32 through Booth sharing.

C. Proposed Multi-Row Interleaved Operation Ordering

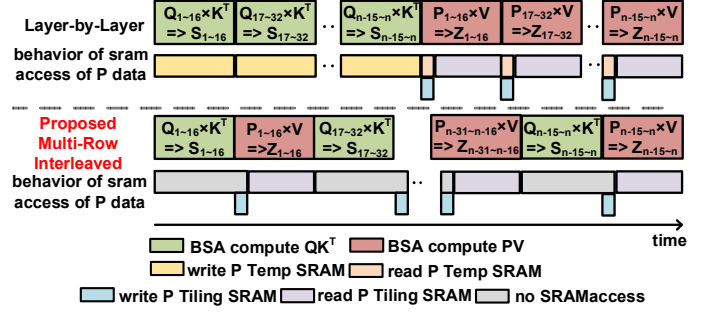


Fig. 8. Comparison between layer-by-layer and proposed Multi-Row Interleaved operation ordering.

Fig.8 shows the proposed Multi-Row Interleaved operation ordering and compares it with layer-by-layer operation ordering, where n is the token length. The P SRAM in Fig.2 includes both the P Temp SRAM and P Tiling SRAM in Fig.8.

Multi-Row Interleaved means that multiple rows of the S matrix and the Z matrix are processed alternately using the same array. Specifically, in the top-level architecture of Fig.5, the BSA is first used to compute the $Q \times K^T$, and the 16 rows of Q are written into the Q Tiling SRAM. The row mux selects the Q data from the Q Tiling SRAM, and the col mux selects the K data from the K SRAM. BSA computes $Q \times K^T$ and outputs the 16 rows of S, which enter the Softmax. When the computation of 16 rows of S is completed, The 16 rows of P from Softmax will also be ready. The row mux selects the P data from Softmax, and the col mux selects the V SRAM. Then, the BSA jumps from the $Q \times K^T$ to the $P \times V$, and P data will be written to the P Tiling SRAM to provide the subsequent data for P Tiling. When the computation of 16 rows of Z is completed, the BSA jumps back from the $P \times V$ to the $Q \times K^T$. Therefore, $Q \times K^T$ and $P \times V$ are executed interleaved until the entire computation is completed. Multi-Row Interleaved significantly reduces the E_{p_sram} , as it eliminates the access overhead of P Temp SRAM compared to layer-by-layer. Typically, the access overhead of P Temp SRAM is higher than that of P Tiling SRAM due to its larger capacity. Specifically, the capacity of P Temp SRAM and P Tiling SRAM are $n \times n \times 8$ bits, and $16 \times n \times 8$ bits, respectively.

IV. EVALUATION

A. Experiment Setup

We evaluate the proposed hardware architecture and compare it with previous works in 28nm technology. The clock

TABLE I. SETTINGS OF SRAMS IN $Q \times K^T$

SA / SRAM	Q	Q Tiling	K	S
IS-SA	32KB (2048×128)	none	32KB (1024×256)	32KB (512×512)
WS-SA	32KB (1024×256)	none	32KB (2048×128)	32KB (512×512)
OS-SA	32KB (2048×128)	1KB (64×128)	32KB (1024×256)	none
Proposed IOM-BSA	32KB (1024×256)	1KB (32×256)	32KB (1024×256)	none

TABLE II. SETTINGS OF SRAMS IN $P \times V$

SA / SRAM	P	P Tiling	V	Z
IS-SA	256KB (16384×128)	none	32KB (1024×256)	4KB (64×512)
WS-SA	256KB (8192×256)	none	32KB (2048×128)	32KB (512×512)
OS-SA	256KB (2048×128)	8KB (512×128)	32KB (1024×256)	none
Proposed IOM-BSA	256KB (8192×256)	8KB (256×256)	32KB (1024×256)	none

frequency of all architectures is fixed at 500MHz and the power supply voltage is fixed at 0.9V. The energy consumption of the multipliers, adders, SRAMs, and registers is obtained using a 28nm Standard Cell Library, commercial SRAM compiler and Synopsys Design Compiler 2013. The utilization of different architectures is determined by dividing the ideal computing cycles by the actual computing cycles. To compare the energy efficiency fairly, we fixed the number of multipliers and adders to 512, and the data format of the multiplier and accumulator are INT8 and INT32. The evaluation model is BERT-base. For the BERT-based model, the token length of 512 can cover most applications [12], so evaluation under the token length $n=512$. TABLE I and II shows the settings of SRAMs for evaluating total energy of SRAM (E_{sram}) in different SAs, including capacity and shape. The format of the shape is depth×width. The array shape of IS/WS/OS-SA is set to 32×16 [10][14][12], and all SAs use data tiling as much as possible to reduce the E_{sram} .

B. The Evaluation of Proposed BSA

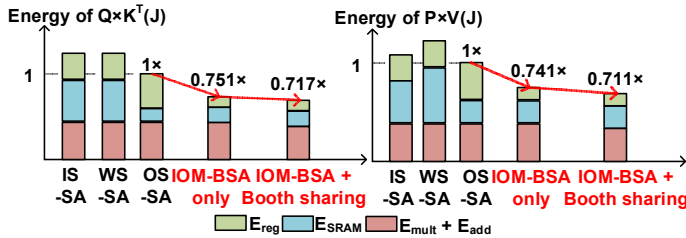


Fig. 9. Evaluation of the energy of BSA.

Fig.9 compared the energy of BSA to conventional SAs in Background. The energy of multiplier E_{mult} and the energy of adder E_{add} are equal since using the same data format and arithmetic circuit. In BSA, IOM-BSA and Booth sharing are techniques to improve energy efficiency. The E_{sram} of IOM-BSA is close to that of OS-SA and better than IS/WS-SA as it takes advantage of outer product. The total register energy E_{reg} of

IOM-BSA is better than that of IS/WS/OS-SA since inner product and broadcast significantly reduce the overhead of accumulators and registers for systolic. IOM-BSA achieves a maximum energy reduction of 25.9% ($0.741 \times$ in Fig.8) compared to the SA with the smallest total energy ($E_{\text{sram}} + E_{\text{reg}} + E_{\text{mult}} + E_{\text{add}}$). Booth sharing further reduces E_{mult} , lowering the total energy by an additional 4%. As the SA scales, larger arrays will amplify the benefits of Booth sharing.

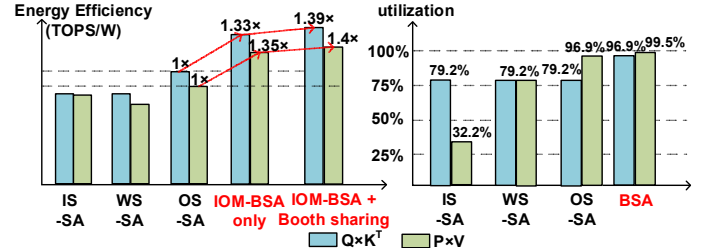


Fig.10. Evaluation of the energy efficiency and utilization of BSA.

The left side of Fig.10 compares the energy efficiency of BSA with conventional SAs. The proposed IOM-BSA achieves up to a 35% improvement in energy efficiency compared with OS-SA, with Booth sharing providing an additional 5% boost. The right side of Fig.9 compares the utilization of BSA with conventional SAs. The high utilization of the proposed BSA is attributed to two factors: 1) tiled broadcast reduces the number of systolic cycles, and 2) the bypass register overlaps offloading and computation, allowing BSA to maintain the highest utilization in both $Q \times K^T$ and $P \times V$.

C. Comparison with Previous Hardware Architecture

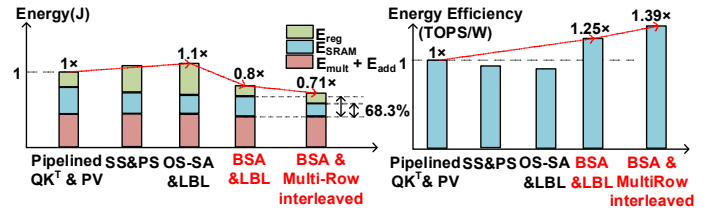


Fig.11. Evaluation of the energy of the proposed architecture.

The hardware architecture and Multi-Row Interleaved operation ordering are evaluated in this section. We compare five attention accelerator hardware architectures: three of them are pipelined QK^T & PV, SS & PS, and OS-SA & layer-by-layer (OS-SA & LBL) in Fig.2. The fourth architecture is BSA & LBL, used to evaluate the improvement brought by BSA, and the fifth architecture is BSA & Multi-Row Interleaved, used to evaluate the improvement brought by Multi-Row Interleaved. In the pipelined QK & PV architecture, we follow the design from [7], where $Q \times K^T$ uses an input stationary matrix-vector multiplication (IS-MVM) array with a shape of 64×4 , and $P \times V$ uses a Broadcast-based outer product array with a shape of 4×64 . The array shape of SS & PS is 32×16 [8][10]. In the OS-SA & layer-by-layer (LBL), the shape of OS-SA is 32×16 [12].

The left side of Fig.11 evaluates the energy of different architectures. OS-SA & LBL has the lowest E_{sram} , as both $Q \times K^T$

and $P \times V$ use the outer product, but the overhead from a large number of registers results in the highest total energy. Although the E_{sram} of pipelined QK^T & PV architecture is the highest, it does not incur the overhead of registers for systolic, which greatly reduces E_{reg} , leading to lower total energy compared to SS & PS and OS-SA & LBL. BSA reduces total energy by 20% compared to the pipelined QK^T & PV architecture, which has the lowest energy in previous work. Multi-Row Interleaved eliminates the access overhead of P Temp SRAM, reducing E_{sram} to 68.3% of the E_{sram} of layer-by-layer. BSA & Multi-Row Interleaved reduces the total energy by 29%. The right side of Fig.11 evaluates the energy efficiency of different architectures. BSA improves the energy efficiency by 25%. Multi-Row Interleaved further improves the energy efficiency by 14%.

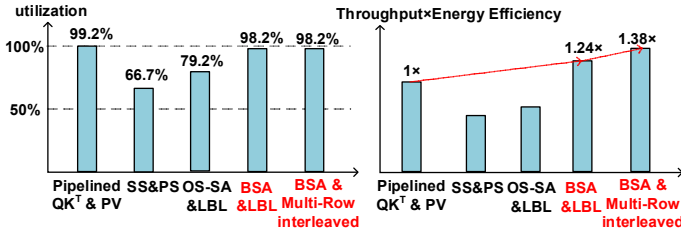


Fig. 12. Evaluation of utilization and throughput×energy efficiency of the proposed architecture.

The left side of Fig. 12 shows the utilization of the five architectures. The pipelined QK^T & PV achieves the highest utilization, reaching 99.2%. SS & PZ has the lowest utilization due to idle cycles of the multiplier caused by pipeline filling, which account for a significant proportion of the $P \times V$. OS-SA & LBL also exhibit low utilization due to the low utilization of the OS systolic array. BSA & LBL can maintain a high utilization of 98.2% due to the high utilization of BSA. Multi-Row Interleaved does not reduce the utilization compared to layer-by-layer while increasing energy efficiency, as it does not introduce idle cycles. Consequently, BSA & LBL architecture and BSA & Multi-Row architecture Interleaved achieve the same high utilization. With improved overall utilization and energy efficiency, the proposed architecture achieves a 38% improvement in throughput×energy efficiency compared to the previous architecture.

V. CONCLUSION

This paper proposes a hardware architecture for the attention mechanism in Transformer, focusing on optimizing energy efficiency and utilization solely through hardware design. The proposed BSA balances data reuse, register saving, and utilization, achieving up to a 40% improvement in energy efficiency compared to conventional SAs while maintaining 99.5% utilization. The proposed Multi-Row Interleaved reduces E_{sram} by 31.7% and further enhances energy efficiency by 14%. By combining the two techniques, the attention accelerator proposed in this paper achieves a 39% improvement in energy efficiency and a 38% improvement in throughput×energy efficiency compared to previous works.

Acknowledgements This work was supported by the NSFC Grant 62404249, 92364202 and 62488101.

REFERENCES

- [1] A. Vaswanet et al., "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [2] J. Devlin et al., "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [3] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI blog*, vol. 1, no. 8, p. 9, 2019.
- [4] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.
- [5] T. J. Ham et al., "A³: Accelerating attention mechanisms in neural networks with approximation," in *2020 IEEE HPCA*. IEEE, 2020, pp. 328–341.
- [6] T. J. Ham et al., "Elsa: Hardware-software co-design for efficient, lightweight self-attention mechanism in neural networks," in *2021 ISCA*. IEEE, 2021, pp. 692–705.
- [7] H. Wang et al., "Spatten: Efficient sparse attention architecture with cascade token and head pruning," in *2021 HPCA*. IEEE, 2021, pp. 97–110.
- [8] Y. Wang et al., "An Energy-Efficient Transformer Processor Exploiting Dynamic Weak Relevances in Global Attention," *IEEE JSSC*, 2023.
- [9] H. You et al., "ViTCoD: Vision Transformer Acceleration via Dedicated Algorithm and Accelerator Co-Design," in *2023 HPCA*. IEEE, 2023, pp. 273–286.
- [10] L. Liu et al., "Sanger: A Co-Design Framework for Enabling Sparse Attention using Reconfigurable Architecture," in *2021 MICRO*. pp. 977–991.
- [11] G. Shen et al., "SALO: An Efficient Spatial Accelerator Enabling Hybrid Sparse Attention Mechanisms for Long Sequences," in *2022 DAC*. pp. 571–576.
- [12] Y. Qin et al., "FACT: FFN-Attention Co-optimized Transformer Architecture with Eager Correlation Prediction," in *2023 ISCA*. pp. 1–14.
- [13] Z. Wang et al., "COSA: Co-Operative Systolic Arrays for Multi-head Attention Mechanism in Neural Network using Hybrid Data Reuse and Fusion Methodologies," in *2023 DAC*. pp. 1–6.
- [14] S. Shanmuga Sundaram et al., "FreFlex: A High-Performance Processor for Convolution and Attention Computations via Sparsity-Adaptive Dynamic Frequency Boosting," *IEEE JSSC*, 2023.
- [15] Y. -H. Chen et al., "Eyeriss: An Energy-Efficient Reconfigurable Accelerator for Deep Convolutional Neural Networks," *IEEE JSSC*, 2017.
- [16] X. Chen et al., "Communication Lower Bound in Convolution Accelerators," in *2020 HPCA*. pp. 529–541.
- [17] Bitu Rouhani et al., "Microscaling Data Formats for Deep Learning," *arXiv preprint arXiv:2310.10537*, 2023.
- [18] Z. Zhang et al., "SpArch: Efficient Architecture for Sparse Matrix Multiplication," in *2020 HPCA*. pp. 261–274.